Massively Parallel Incremental Betweenness Centrality Algorithms for Weighted Graphs

Chilei Wang, Qiang-Sheng Hua*, and Hai Jin

National Engineering Research Center for Big Data Technology and System/Services Computing Technology and System Lab/Cluster and Grid Computing Lab, School of Computer Science and Technology, Huazhong University of Science and Technology, Wuhan 430074, People's Republic of China.
qshua@hust.edu.cn

Abstract. We study the incremental betweenness centrality (BC) problem for weighted graphs in the Massively Parallel Computation (MPC) model, where each machine has $O(n^\alpha)$ memory for a constant $\alpha \in (0, 1)$. When preprocessing an n-node m-edge graph, we replace the forward and backward passes of the parallel Brandes algorithm with the blocked Floyd-Warshall algorithm and vector operations. This reduces the round complexity from $O(n/\alpha)$ to $O(n^{1-\alpha/2}/\alpha)$ and the memory complexity from $O(n^2 + mn)$ to $O(n^2)$. We consider three types of incremental updates: (i) single-edge insertion or weight decrease; (ii) single-vertex insertion with incident edges; (iii) multiple-edge insertion or weight decrease. For (i) and (ii), we design efficient algorithms based on the blocked Floyd-Warshall algorithm framework and vector operations, achieving constant rounds. For (iii), we propose a parallel multi-edge update algorithm with $O(n^{3-\alpha/2})$ total memory, using the framework of blocked Floyd-Warshall algorithm and parallel matrix multiplication. The round complexity required is $O(\alpha^{-1})$ when the number of involved vertices $z \leq n^{\alpha/2}$, and is $O(z/(n^{\alpha/2}\alpha))$ when $z > n^{\alpha/2}$. In summary, our incremental algorithms significantly reduce the round complexity required for recomputation.

Keywords: Betweenness Centrality · Massively Parallel Computation Model · Round Complexity · Algebraic Methods · Incremental Algorithm.

1 Introduction

In the past decade, the study of graph algorithms in *the Massively Parallel Computation Model (the MPC model)* [12] has received extensive attention, such as graph connectivity [3], minimum cut [9], and shortest paths [8, 14]. However, most of these parallel graph algorithms are designed for static graphs.

In practice, real-world graphs are often dynamically evolving. When updates such as edge (or vertex) deletions or insertions occur, applying static graph algorithms necessitates recomputing the entire input graph from scratch. This would cause unnecessary computation and time overhead, especially when the changes

* Corresponding author(qshua@hust.edu.cn).

to the graph only affect a fraction of the final results. *Dynamic graph algorithms* process these evolving graphs more efficiently than their static counterparts, both on a single machine [20] and in the parallel models [7, 22].

Although work on dynamic graph algorithms in the MPC model remains relatively limited [7, 22, 10, 16], these studies have achieved significantly lower round complexities compared to their static parallel counterparts. For instance, it costs only $O(1)$ rounds to update the minimum spanning tree in the MPC model, but the round complexity required for the static counterpart is $O(\log n)$ [16].

Despite these advances, the dynamic MPC algorithms are largely confined to structural problems, like connectivity [16] and shortest paths [28]. More complex metrics, particularly *Betweenness Centrality (BC)*, remain unexplored in the MPC model. BC is a primary metric in network analysis for quantifying the importance of nodes. It is defined as the ratio of the shortest-path count through a node to the total count of shortest paths in the graph [15]. Nodes with high BC values play a critical role in a network, with applications in bioinformatics, social network analysis, and cybersecurity [2, 27]. Other relevant centrality measures include graph centrality, closeness centrality, and stress centrality, all of which are closely tied to shortest paths [5, 15]. Among them, graph and closeness centralities can be directly derived from pairwise distances, whereas computing BC and stress centrality requires both the distances and shortest-path counts [1].

In the RAM (Random Access Memory) model, the Brandes algorithm computes BC exactly in $O(mn)$ time [5]. Although it has been extended to parallel and distributed models, such as the Congest model [15] and PRAM (Parallel RAM) model [27, 2], these implementations often require high round complexity or large space overhead, which limits the scalability.

While dynamic BC algorithms have been studied in the single-machine model, most are restricted to amortized time complexity analysis [21, 23]. The existing parallel dynamic BC algorithms focus on unweighted graphs and are based on empirical evaluations [26, 17]. This is because all existing BC algorithms, static or dynamic, sequential or parallel, rely on the framework of Brandes algorithm. The predecessor DAG maintenance and backward dependency propagation make it difficult to achieve low round complexity in parallel models [17], and incurs extensive recomputation triggered by layer-by-layer updates in dynamic scenarios [18, 20], thus hindering substantial theoretical progress.

1.1 The Contributions

In this paper, we study the dynamic BC problem in the MPC model. The aim is to design efficient MPC algorithms that update the BC values for all nodes in a weighted graph under incremental updates. Specifically, we consider the following types of incremental operations: i) single edge insertion or weight decrease; ii) single vertex insertion with its incident edges; iii) multiple edge insertions or weight decreases. Instead of using the framework of Brandes algorithm, we adopt the algebraic approaches and will demonstrate that the algebraic methods can update the BC values of nodes in constant rounds for all these update operations. Our primary contributions are listed below:

• We propose a parallel BC algorithm for weighted graphs via the blocked Floyd-Warshall algorithm and vector operations, with $O(n^{1-\alpha/2}/\alpha)$, improving over the parallel Brandes algorithm with $O(n/\alpha)$ rounds. (see Rows 2 - 3 of Table 1 and details in Section 5.1). $\alpha \in (0, 1]$ is the memory parameter.

• Utilizing the framework of blocked Floyd-Warshall algorithm, vector operations, and parallel matrix multiplication, we design efficient parallel incremental BC algorithms for three update cases (see Rows 4 - 6 in Table 1 and details in Section 5.2). These algorithms greatly reduce the round complexity compared to that of recomputation.

Following this, we introduce our main results in Theorem 1 and Table 1. The proof of Theorem 1 follows readily from Lemmas 2, 3, 4, 5, and 6.

Theorem 1. *For an n-vertex weighted graph G and the initial matrix DC (defined as in (1) of Section 5), Algorithm 1 computes the distance and shortest-path count matrices in $O(n^{1-\alpha/2}/\alpha)$ rounds ($\alpha \in (0, 1)$) and $O(n^2)$ total memory.*

For single-edge update, Algorithm 4 updates the matrix DC in $O(1/\alpha)$ rounds. For single-vertex updates, Algorithm 5 updates the matrix DC correctly in $O(1/\alpha)$ rounds. For multiple-edge updates ($N(z)$ inserted edges), Algorithm 7 updates the matrix DC correctly in $O(\alpha^{-1})$ rounds with $O(n^2)$ memory when $z \leq n^{\alpha/2}$ and $O(z/(n^{\alpha/2}\alpha))$ rounds with $O(n^{3-\alpha/2})$ memory for $z > n^{\alpha/2}$.

Using DC computed or updated above, Algorithm 8 computes the BC values for all nodes using $O(1/\alpha)$ rounds and $O(n^{3-\alpha/2})$ totla memory.

Table 1. Complexity Comparisons between the Parallel Brandes Algorithm and Our Parallel Incremental BC Algorithms

Algorithm	Round Complexity	Total Memory	Update Type
Preprocessing (static)			
SPBC	$O(n/\alpha)$	$O(n(n+m))$	No
Algorithm 1 and Algorithm 8	$O(n^{1-\alpha/2}/\alpha)$	$O(n^{3-\alpha/2})$	No
Incremental updates			
Algorithm 4 and Algorithm 8	$O(1/\alpha)$	$O(n^{3-\alpha/2})$	Single-edge updates
Algorithm 5 and Algorithm 8	$O(1/\alpha)$	$O(n^{3-\alpha/2})$	Single-vertex updates
Algorithm 7 and Algorithm 8	$O(z/(n^{\alpha/2}\alpha))$	$O(n^{3-\alpha/2})$	Multiple-edge updates

Note: z represents the number of distinct vertices incident to the updated edges. When $z \leq n^{\alpha/2}$, handling multiple edge updates requires only constant rounds.

2 Related Work

Most existing dynamic BC algorithms follow the framework of the Brandes algorithm. This algorithm consists of Forward Pass(computes distances and shortest-path counts) and Backward Pass (accumulates dependencies to determine the BC values for vertices).

Lee et al. [19] proposed the first dynamic BC algorithm for edge updates, combining biconnected components with the Brandes algorithm. Subsequently, Singh et al. [11] extended this approach to vertex updates. Green et al. [13] designed a BC algorithm for streaming graphs with edge insertions, which maintains a BFS (Breadth-First Search) tree for each vertex. All three works focus on unweighted graphs. Kas et al. [18] further studied the incremental weighted BC problem, using the dynamic APSP (All Pairs Shortest Paths) algorithm of [24]. All these approaches achieve good empirical performance, but with no theoretical improvement over the Brandes algorithm in the worst case.

The first breakthrough came from Nasre et al. [20], who maintained SSSP (Single-Source Shortest Paths) DAGs in weighted graphs for single edge or single vertex insertions. Their incremental algorithm takes $O((\bar{m}^* + m_v^*)n + n^2)$ time, which improves upon the Brandes algorithm. m_v^* denotes the number of distinct edges on the shortest paths passing through the vertex v and $\bar{m}^*$ is the average of m_v^* over all vertices. Then, Nasre et al. [21] studied the weighted BC problem under vertex deletions. Their decremental algorithm builds on the fully dynamic APSP algorithm of [6] and achieves $O(v^{*2} \log n)$ amortized time, where v^* is the maximum number of distinct edges on shortest paths through any single vertex. In the worst case, this time complexity matches that of the Brandes algorithm. Building on this line of work, Pontecorvi and Ramachandran [23] extended the approach to the fully dynamic setting with vertex updates. They designed a sequential dynamic algorithm with $O(v^{*2} \log^3 n)$ amortized time.

However, research on parallel dynamic BC algorithms is limited, which focuses on shared-memory models and the unweighted graphs. These works adopt the approach of Lee et al. [19], which localizes computation within affected graph components to update BC values efficiently. For example, the iCentral algorithm presented by Jamour et al. [17] and the Batch-iCentral algorithm of Shukla et al. [26] handle the single-edge and multiple-edge updates, respectively.

3 The MPC Model and Problem Definitions

3.1 The MPC Model

We consider the MPC model in the setting of strongly sublinear memory. Namely, each processor has $O(n^\alpha)$ memory, where n represents the input size of a given graph and $\alpha \in (0, 1)$ is a constant. The input size N, the memory size S for each processor, and the number of processors P satisfy the relation that $O(N) = P \cdot S$.

In the MPC model, we compute the input graph in synchronous rounds. In each round, every processor executes local computation without inter-processor communication. Then, processors exchange data with each other, which is used for computation in the next round. The data size received or sent by each processor is at most S. Moreover, the processors are interconnected in pairs.

Complexity Measures: The main objective of the MPC model is to minimize the round complexity. Our second objective is to make the total memory required as small as possible.

Some commonly used MPC subroutines, such as broadcast (reduction) operations [8], sorting [12], and finding the maximum or minimum among n elements [8], can all be completed in $O(1/\alpha)$ rounds.

3.2 Problem Definitions

In this paper, we consider a graph $G = (V, E)$ with positive real edge weights, where $V(|V| = n)$ and $E(|E| = m)$ represent the vertex and edge sets, respectively. For any two vertices $x, y \in V$, $d(x, y)$ denotes the distance between them. $w(x, y)$ is the weight of the edge $(x, y) \in E$.

We now give the formal mathematical definition of BC below.

Definition 1 (Betweenness Centrality). *The betweenness centrality of a vertex $u \in V$ is defined as*

$$C_B(u) = \sum_{\substack{x,y \in V \\ x \neq y,\, x \neq u,\, y \neq u}} \frac{\sigma_{xy}(u)}{\sigma_{xy}},$$

where σ_{xy} is the count of shortest paths from x to y, and $\sigma_{xy}(u)$ is the number of those paths that pass through u.

In addition to betweenness centrality, our algorithmic framework also supports the update of the centralities mentioned in Section 1.

Definition 2 (Other Centralities). *Given the graph $G = (V, E)$ above, for any vertex $u \in V$, the closeness centrality $C_C(u)$, the graph centrality $C_G(u)$, and the stress centrality $C_S(u)$ are defined as follows:*

$$C_C(u) = \frac{1}{\sum_{y \in V} d(u, y)}, \quad C_G(u) = \frac{1}{\max_{y \in V} d(u, y)}, \quad C_S(u) = \sum_{\substack{x,y \in V \\ x \neq u,\, y \neq u, x \neq y}} \sigma_{xy}(u).$$

4 Techniques

We first introduce the Brandes algorithm and the sequential incremental BC algorithm proposed by Nasre et al. [20] in Section 4.1. Then, Section 4.2 describes our parallelization of the Brandes algorithm in the MPC model and analyses its complexities. Finally, Section 4.3 discusses the challenges of parallelizing Nasre et al.'s incremental BC algorithm and proposes our strategies for these challenges.

4.1 The Brandes Algorithm and the Sequential Incremental BC algorithm of Nasre et al. [20]

Our algorithm is motivated by the sequential incremental BC algorithm of Nasre et al. [20], which relies on the Brandes algorithm. The Brandes algorithm consists of a forward pass and a backward pass.

Forward pass: computing the distances and shortest-path counts. For a vertex $u \in V$ and a source $x \in V$, we define u's predecessor set $P_x(u) = \{v \in V \mid (v,u) \in E, d(x,u) = d(x,v) + w(v,u)\}$. Then, using the Breadth First Search (BFS) algorithm from each source x, we calculate the count of shortest paths from x to $y \in V$ by $\sigma_{xy} = \sum_{v \in P_x(y)} \sigma_{xv}$.

Backward pass: computing the dependencies and BC values. The dependency $\delta_{x\cdot}(u)$ of a source x on the vertex $u \in V$ is defined as $\delta_{x\cdot}(u) = \sum_{y \in V} \delta_{xy}(u)$, where $\delta_{xy}(u) = \frac{\sigma_{xy}(u)}{\sigma_{xy}}$ represents the pair dependency of a pair x, y on u. Brandes [5] computes $\delta_{x\cdot}(u) = \sum_{v:u \in P_x(v)} \frac{\sigma_{xu}}{\sigma_{xv}}(1 + \delta_{x\cdot}(v))$ in reverse BFS order. The BC value for u is then $C_B(u) = \sum_{x \neq u} \delta_{x\cdot}(u)$.

Nasre et al. [20] employ the Brandes algorithm for preprocessing. Specifically, each source maintains a directed acyclic graph (DAG) initially. Unlike the shortest path tree, where each node stores only one predecessor, each node in the DAG of a given source stores all its predecessors.

In the update phase, they update the forward pass and recompute the backward pass of the Brandes algorithm. For any $x, y \in V$, let $d^{new}(x,y)$ and $d^{old}(x,y)$ denote the distance between x and y after and before the update of the edge (v,u), respectively. σ_{xy}^{new} and σ_{xy}^{old} are defined analogously. The new weight of (v,u) is denoted by $w^{new}(v,u)$. When the weight of (v,u) decreases, the distance between any two nodes x and y can be easily updated by

$$d^{new}(x,y) = d^{old}(x,v) + w^{new}(v,u) + d^{old}(u,y).$$

Since the distances from any node x to v or from u to any node y remain unchanged, the shortest-path counts are then updated by the three cases below:

1. If $d^{new}(x,y) > d^{old}(x,y)$, then $d^{new}(x,y) = d^{old}(x,y)$ and $\sigma_{xy}^{new} = \sigma_{xy}^{old}$;
2. If $d^{new}(x,y) = d^{old}(x,y)$, then $\sigma_{xy}^{new} = \sigma_{xy}^{old} + \sigma_{xv}^{old} \cdot \sigma_{uy}^{old}$;
3. If $d^{new}(x,y) < d^{old}(x,y)$, then $\sigma_{xy}^{new} = \sigma_{xv}^{old} \cdot \sigma_{uy}^{old}$.

With the new distances and shortest-path counts, Nasre et al. [20] updated the DAG (i.e., the incoming edges from the predecessors) for each source. Then, they complete the forward pass. For the backward pass, they recompute the dependencies δ_{xy} for all pairs of nodes to obtain the final BC values, using the updated DAGs. The sequential incremental algorithm runs in $O((\bar{m}^* + m_v^*)n + n^2)$ time, as discussed in Section 2.

In addition, Nasre et al. [20] extend single-edge updates to single-vertex updates, using a similar idea. However, for an undirected graph, their methods must be executed twice.

4.2 The Straightforward Parallel Brandes Algorithm (SPBC)

Now, we introduce the straightforward parallel Brandes algorithm in the MPC model. In the forward pass, we execute the restricted Bellman-Ford algorithm [8] for $O(n/\alpha)$ rounds, which traverses all neighbors of every vertex in each iteration. Similarly, the backward pass also takes $O(n/\alpha)$ rounds to compute dependency

values for each vertex, traversing all predecessors of a vertex per iteration. During each iteration, inter-machine communication incurs $O(1/\alpha)$ rounds because of the broadcast, reduction, sorting, and find maximum operations. The number of iterations is bounded by $O(n)$, as any shortest path in the graph consists of at most $n-1$ edges.

To reduce the overall round complexity, we replicate the graph n times, one copy per source. This allows the forward and backward passes for all sources to be executed in parallel, at the cost of $O(n(n+m))$ total memory (see Algorithm 3 in Appendix A. Finally, we compute the BC values for all nodes via a broadcast operation. The detailed analysis of SPBA is provided in Appendix A. In summary, SPBA takes $O(n/\alpha)$ rounds.

4.3 Challenges and Strategies

When an edge's weight decreases, the forward pass of the sequential incremental BC algorithm (see Section 4.1) can be updated in $O(\alpha^{-1})$ rounds. Specifically, for each source x, we broadcast d_{xv}^{old} and σ_{xv}^{old} to the machines that store all nodes in the DAG of x in $O(\alpha^{-1})$ rounds. Similarly, we broadcast d_{uy}^{old}, the new edge weight $w^{new}(v,u)$, and σ_{uy}^{old} to the machines that store every node y (except the source) in the DAGs of all sources, which takes $O(\alpha^{-1})$ rounds. The forward pass can then be updated locally. However, since the depth of each DAG is at most n, recomputing the backward pass of the incremental BC algorithm requires $O(n/\alpha)$ rounds. Consequently, in the MPC model, the straightforward parallel incremental BC algorithm is no better than SPBA.

The key challenge lies in the strict layer-by-layer dependency in the backward pass. A single vertex update involves at most n edges. The information of n edges exceeds the memory of a single machine and communication conflict between machines can occur. As a result, the round complexity of the forward pass becomes unpredictable. In addition, the value σ_{xy} can reach $n^{O(n)}$ and requires $O(n \log n)$ bits, since the number of edges on the shortest path from x to y is at most $n-1$ in weighted graphs. This may cause both local memory and communication beyond standard matrix multiplication, as each processor's memory in the MPC model is limited. Namely, the memory utilization is inefficient.

To overcome these challenges, we employ algebraic methods to compute and update the BC values for all nodes. Specifically, we improve the blocked Floyd-Warshall algorithm in [25] to preprocess an initial graph and obtain the distances and shortest-path counts between nodes. For the backward pass, we bypass the computation of δ_{xy} and directly compute the BC values of all nodes, by combining the path-decomposition strategy with vector-vector operations, which can be completed in constant rounds.

For single-edge or single-vertex updates, following the approach of [28], we update the distances and shortest-path counts in constant rounds, using the framework of the blocked Floyd-Warshall algorithm. We further extend our method to handle multiple-edge updates.

To solve the large-value problem, we represent σ_{xy} in logarithmic form as $l_{xy} = \log_2(\sigma_{xy})$ (refer to Appendix D). Since $\sigma_{xy} \in [1, n^n]$, it follows that

$\log_2(\sigma_{xy}) \in [0, n \log n]$. As a result, each $\log_2(\sigma_{xy})$ requires only $O(\log n)$ bits for storage, thereby maximizing memory utilization. Moreover, the processors exchange l_{xy} instead of σ_{xy}, allowing more values to be transmitted per round.

5 The Parallel Incremental Betweenness Centrality Algorithm

First, Section 5.1 presents an improved blocked Floyd-Warshall algorithm that replaces the forward pass of SPBA, reducing the round complexity of preprocessing the initial graph. Section 5.2 then designs the update algorithms for the distance matrix and the shortest-path count matrix under three types of incremental updates introduced in Section 1.1. As noted in Section 4.3, the algebraic methods compute the BC values for all nodes in constant rounds once the distance and shortest-path count matrices are known; thus, for each update, we recompute the BC values, which will be detailed in Section 5.3.

5.1 The Improved Block Floyd-Warshall Algorithm

We first introduce a matrix DC that contains both the distances and shortest-path counts, facilitating efficient computation. Specifically,

$$DC = \begin{bmatrix} (D[1,1], \sigma_{11}) & \cdots & (D[1,n], \sigma_{1n}) \\ (D[2,1], \sigma_{21}) & \cdots & (D[2,n], \sigma_{2n}) \\ \vdots & \ddots & \vdots \\ (D[n,1], \sigma_{n1}) & \cdots & (D[n,n], \sigma_{nn}) \end{bmatrix} \tag{1}$$

For matrix DC: we set $D[x,x] = 0$ and $\sigma_{xx} = 1$ for all $x \in V = \{1, 2, \cdots, n\}$; $D[x,y] = A[x,y]$ and $\sigma_{xy} = 1$ for all $(x,y) \in E$; for two distinct nodes $x, y \in V$ and $(x,y) \notin E$, $D[x,y] = +\infty$ and $\sigma_{xy} = 0$.

Algorithm 1 presents the pseudocode of the improved blocked Floyd-Warshall algorithm. First, the matrix DC is partitioned into $O(n^{1-\alpha})$ submatrices with size $n^{\alpha/2} \times n^{\alpha/2}$, resulting in $O(n^{1-\alpha/2})$ iterations (refer to line 1). In the k-th iteration, line 2 updates the diagonal submatrix DC_{kk} using Algorithm 2. Lines 3 and 4 then broadcast DC_{kk} and $Dist_{kk}$ to the processors storing the k-th row or the k-th column submatrices. Lines 5 - 8 execute local computation in parallel to update these row and column submatrices. Lines 9 and 10 broadcast the updated row and column submatrices to the processors storing the remaining submatrices. Finally, lines 11 - 13 update the remaining submatrices without communication.

Next, we show the pseudocode of the sequential Floyd-Warshall algorithm in Algorithm 2, which serves as a key subroutine in our improved blocked Floyd-Warshall algorithm (see lines 2, 6 - 7, and 10 of Algorithm 1).

Now, we analyze the complexities of Algorithm 1 below:

Lemma 1. *For an n-vertex weighted graph G and the initial matrix DC, Algorithm 1 computes the distance matrix and shortest-path count matrix in $O(n^{1-\alpha/2}/\alpha)$ rounds ($\alpha \in (0,1)$) and $O(n^2)$ total memory.*

Algorithm 1 The Blocked Floyd-Warshall Algorithm for Distances and Shortest-path counts

Input: The $n \times n$ matrix DC with n^2 tuples and an $n \times n$ empty matrix $Dist$, which are distributed among $n^{2-\alpha}$ processors $P_{i,j}$ $(i, j \in \{1, 2, \cdots, n^{1-\alpha/2}\})$.
Output: The matrix DC that stores the distances and shortest-path counts.

for $k = 1, 2, \cdots, n^{1-\alpha/2}$ **do**
  Execute SequFW$(DC_{kk}, DC_{kk}, DC_{kk}, Dist_{kk}, n^{\alpha/2})$ 2 ▷ refer to Algorithm 2 and update the k-th row and the k-th column submatrix $DC_{kk} = (D_{kk}, C_{kk})$
  The processor $P_{k,k}$ broadcast DC_{kk} and $Dist_{kk}$ with size $n^{\alpha/2} \times n^{\alpha/2}$ to the processors $P_{i,k}$ that store DC_{ik} and $Dist_{ik}$, where $i \in \{1, 2, \cdots, n^{1-\alpha/2}\}$
  The processor $P_{k,k}$ broadcast DC_{kk} and $Dist_{kk}$ with size $n^{\alpha/2} \times n^{\alpha/2}$ to the processors $P_{k,i}$ that store DC_{ki} and $Dist_{ki}$
  for all $i \in \{1, 2, \cdots, n^{1-\alpha/2}\}$ **do** in parallel
    Execute SequFW$(DC_{kk}, DC_{ki}, DC_{ki}, Dist_{ki}, n^{\alpha/2})$ ▷ Update the submatrices in the k-th row
    Execute SequFW$(DC_{ik}, DC_{kk}, DC_{ik}, Dist_{ik}, n^{\alpha/2})$ ▷ Update the submatrices in the k-th column
  end for
  The processor $P_{i,k}$ broadcast DC_{ik} and $Dist_{ik}$ with size $n^{\alpha/2} \times n^{\alpha/2}$ to the processors $P_{i,j}$ that store DC_{ij} and $Dist_{ij}$, where $i, j \in \{1, 2, \cdots, n^{1-\alpha/2}\}$
  The processor $P_{k,i}$ broadcast DC_{ki} and $Dist_{ki}$ with size $n^{\alpha/2} \times n^{\alpha/2}$ to the processors $P_{i,j}$ that store DC_{ij} and $Dist_{ij}$
  for all $i, j \in \{1, 2, \cdots, n^{1-\alpha/2}\}$ **do** in parallel
    Execute SequFW$(DC_{ik}, DC_{kj}, DC_{ij}, Dist_{ij}, n^{\alpha/2})$ ▷ Update the remaining submatrices
  end for
end for
return DC

Algorithm 2 SequFW$(DC_1, DC_2, DC_3, Dist, n)$

Input: The three $n \times n$ matrices DC_i with n^2 tuples, where $i \in \{1, 2, 3\}$ and $DC_i = \left[D_i[i', j'], \sigma^i_{i'j'}\right]_{i', j' \in \{1, 2, \cdots, n\}}$. An $n \times n$ empty matrix $Dist$.
Output: The updated DC_3.

for $k' = 1, 2, \cdots, n$ **do**
  for $i' = 1, 2, \cdots, n$ **do**
    for $j' = 1, 2, \cdots, n$ **do**
      $Dist[i', j'] = D_1[i', k'] + D_2[k', j']$
      if $Dist[i', j'] < D_3[i', j']$ **then**
        $D_3[i', j'] = Dist[i', j']$ and $\sigma^3_{i'j'} = \sigma^1_{i'k'} \cdot \sigma^2_{k'j'}$
      end if
      if $Dist[i', j'] == D_3[i', j']$ **then**
        $\sigma^3_{i'j'} += \sigma^1_{i'k'} \cdot \sigma^2_{k'j'}$
      end if
    end for
  end for
end for
return DC_3

Proof. In Algorithm 1, only broadcast operations occur in lines 3 - 4 and 9 - 10, which all take $O(\alpha^{-1})$ rounds. With $n^{1-\alpha/2}$ iterations, the round complexity of Algorithm 1 is $O(n^{1-\alpha/2}/\alpha)$. Since the size of matrix DC is $O(n^2)$ and each processor has $O(n^\alpha)$ memory, the total memory required is $O(n^2)$.

Remark. As formalized in Definition 2, with the distance matrix D (computed by Algorithm 1), the closeness and graph centralities of all vertices follow from D via row-wise summation and maximization, respectively, both in $O(1/\alpha)$ rounds.

5.2 The Parallel Incremental BC Algorithms

This section designs three parallel incremental BC algorithms for three types of incremental updates of the weighted BC problem, leveraging the Floyd-Warshall algorithm framework and parallel matrix multiplication.

Single edge insertion or weight decrease As mentioned in Section 4.1, when an edge (v, u) is inserted or the edge weight $w(v, u)$ decreases, there are three types of changes in distances and shortest path counts between nodes. After preprocessing the initial weighted graph with Algorithm 1, we update the matrix DC according to (v, u), as shown in Algorithm 4 of Appendix B.

The idea behind Algorithm 4 for updating the matrix DC is simple. When an edge (v, u) is inserted or its weight decreases, the distances from all nodes to v and from u to all nodes remain unchanged. Hence, we extract two vectors from DC: the $n \times 1$ vector $DC[:, v]$ and the $1 \times n$ vector $DC[u, :]$, which store the distances and shortest-path counts from all nodes to v and from u to all nodes, respectively. We also set an empty matrix DC' with the same size as DC. In the MPC model, we first broadcast $w(v, u)$ to the processors storing $DC[:, v]$; these processors then compute an $n \times 1$ candidate vector $DC'[:, u]$, which store the distances and shortest-path counts from all nodes to u passing (v, u) (implying $\sigma'_{i'u} = \sigma_{i'v}$ in $DC'[:, u]$). Then, lines 2 - 3 broadcast the vectors $DC'[:, u]$ and $DC[u, :]$ to the designated processors. In line 5, all processors compute the outer product of $DC'[:, u]$ and $DC[u, :]$ in parallel. Finally, line 6 updates DC using DC', according to formula (2).

$$DC[i', j'] = \begin{cases} \left(D'[i', j'], \sigma'_{i'j'}\right) & \text{if } D'[i', j'] < D[i', j'], \\ \left(D[i', j'], \sigma_{i',j'} + \sigma'_{i'j'}\right) & \text{if } D'[i', j'] = D[i', j']. \end{cases} \tag{2}$$

Lemma 2. *Given an n-vertex weighted graph G and the matrix DC computed by Algorithm 1, Algorithm 4 updates the matrix DC in $O(1/\alpha)$ rounds. The total memory required is $O(n^2)$.*

Proof. In Algorithm 4, only broadcast operations are executed in lines 1, 3, and 4, all using $O(\alpha^{-1})$ rounds. The outer product of $DC'[:, u]$ and $DC[u, :]$ is an $n \times n$ candidate matrix DC' with n^2 tuples, which results in $O(n^2)$ total memory.

In addition, unlike Nasre's sequential incremental algorithm [20], which runs twice on an undirected graph (for edges (u, v) and (u, v)), Algorithm 4 can

process undirected graphs directly. Since DC is a symmetric matrix, the outer product of $DC'[:,u]$ and $DC[u,:]$ yields the same result as that of $DC[:,u]$ and $DC'[u,:]$.

Single vertex insertion with its incident edges Since a single vertex update involves inserting all its incident edges, we can efficiently handle a single vertex insertion (see Algorithm 5 of Appendix B) in a similar manner to single-edge updates. Specifically, we use two vectors with size n to store the incoming and outgoing edge information of the inserted vertex v, as follows:

$$V^{in} = [d(1,v), d(2,v), \cdots, d(n,v)]^T, V^{out} = [d(v,1), d(v,2), \cdots, d(v,n)].$$

Where $d(i,v) = w(i,v)$ and $d(v,j) = w(v,j)$ for any $i,j \in \{1,2,\cdots,n\}$.

Then, we initialize two vectors as follows, which are used to store the distances and shortest-path counts between v and other nodes:

$$VC^{in} = [(V^{in}[1], \sigma_{1v}), (V^{in}[2], \sigma_{2v}), \cdots, (V^{in}[n], \sigma_{nv})]^T,$$

$$VC^{out} = [(V^{out}[1], \sigma_{v1}), (V^{out}[2], \sigma_{v2}), \cdots, (V^{out}[n], \sigma_{vn})].$$

The only difference between Algorithm 5 and Algorithm 4 is that the former computes two matrix-vector products: (i) $DC \cdot VC^{in}$ (refer to lines 1 - 3), and (ii) $VC^{out} \cdot DC$ (refer to lines 4 - 6). Lines 7 - 10 update DC, which are the same as lines 3 - 7 of Algorithm 4.

Lemma 3. *Given an n-vertex weighted graph G and the matrix DC computed by Algorithm 1, Algorithm 5 updates the matrix DC correctly in $O(1/\alpha)$ rounds. The total memory required is $O(n^2)$.*

Proof. In Algorithm 5, only broadcast operations are executed in lines 1, 4, 7 and 8, each using $O(\alpha^{-1})$ rounds. The outer product of VC^{in} and VC^{out} yields an $n \times n$ candidate matrix DC' with n^2 tuples, hence the total memory is $O(n^2)$.

For correctness of Algorithm 5, with DC computed by Algorithm 1 before inserting v, lines 1 - 6 compute the distances and shortest-path counts between v and other nodes correctly. Specifically, for each $i \in \{1,2,\cdots,n\}$, we have $d(v,i) = \min_{j^{out}(v)}\{w(v,j) + d(j,i)\}$, and $d(j,i)$ for some $j \in \{1,2,\cdots,n\}$ remains unchanged after inserting v. If $d(j,i)$ decreases after inserting v, then $d(j,i) = d(j,v) + d(v,i)$, which leads to a contradiction. Then, with $d(v,i)$ computed, we can update σ_{vi} correctly. Since lines 7 - 10 execute the same operations as Algorithm 4, we can update DC correctly.

Multiple edge insertions or weight decreases Unlike single-edge and single-vertex updates, which require only vector operations, multiple-edge updates involve multiple distinct vertices. Processing k edges sequentially with Algorithm 4 would require k executions, leading to a linear round complexity. Although these edges involve at most $2k$ distinct nodes, this is not accurate enough. So we denote the actual number of involved nodes by z and $N(z)$ as the set of the updated

edges. For the above challenges, we combine the blocked Floyd-Warshall algorithm framework with parallel matrix multiplication to achieve efficient multiple-edge updates (Algorithm 7 in Appendix B).

First, we introduce a key subroutine (Algorithm 6 in Appendix B) of Algorithm 7, based on the known MPC algorithm for solving matrix multiplication [14]. Algorithm 6 can compute the product DC^3 of DC^1 and DC^2, which store the distances and shortest-path counts between nodes.

In Algorithm 6, lines 1 - 2 broadcast submatrices of DC^1 and DC^2 to the designated processors. Then, each processor group of size $O(n^{1-\alpha/2})$ handles an $n^{\alpha/2} \times n^{\alpha/2}$ submatrix of DC^3 in line 3. Finally, in line 4, the processors compute the final DC^3 by a reduction operation. Therefore, Algorithm 6 requires $O(\alpha^{-1})$ rounds and $O(n^{3-\alpha/2})$ total memory (see [14] for details).

Next, we introduce Algorithm 7. In line 1, we update the adjacency matrix A with $N(z)$ update edges, following the conditions below: i) for the $N(z)$ update edges, we use the new weights in the corresponding elements in the matrix A; ii) for the remaining elements in the matrix A, we retain the original weights.

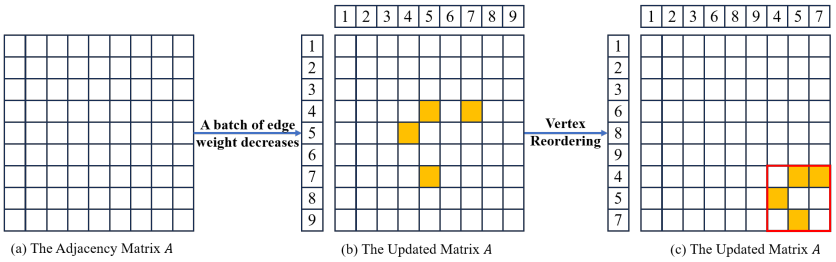


Fig. 1. A concrete example of line 2 in Algorithm 7. (a) shows an adjacency matrix A, each block of which is a submatrix that fits into a single machine. The yellow blocks in (b) are the update edges. (c) shows the updated matrix A after vertex reordering.

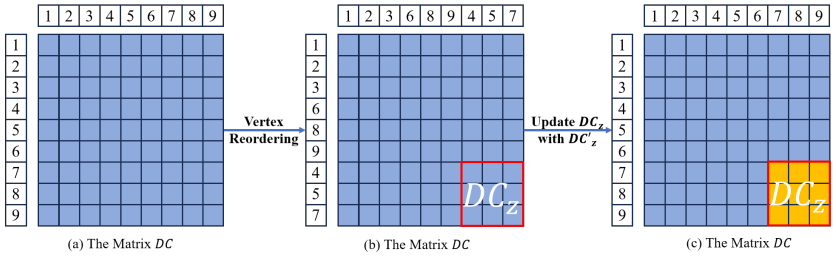


Fig. 2. (a) shows the matrix DC. (b) shows DC after the vertex reordering, which is the same as that of the matrix A. (c) the submatrix DC_z is updated using DC'_z.

Line 2 reorders the indices of vertices in matrix A and DC such that the indices of z affected vertices are arranged in the last z rows and the last z columns (see Fig. 1 and Fig. 2). Then, we update the distances and shortest-path counts between these z vertices, as described in lines 3 - 6 and 13 - 14 of Algorithm 7 (see Fig. 3(a)). When $z \leq n^{\alpha/2}$, we update DC_z on a single processor (lines 5 - 6), using Algorithm 2. We update the rest submatrices (lines 7 - 11) the same as Algorithm 1. For $z > n^{\alpha/2}$, to reduce the round complexity, we partition DC into submatrices with size $z \times z$, which are stored in multiple machines. Then, we update DC_z using Algorithm 1 (lines 13 - 14). Subsequently, according to the framework of Algorithm 1, we update the horizontal and vertical submatrices with size $z \times z$, using D_z in lines 15 - 17 of Algorithm 7 (see Fig. 3(b)). Finally, we update the rest submatrices in lines 18 - 21 of Algorithm 4 (see Fig. 3(c)).

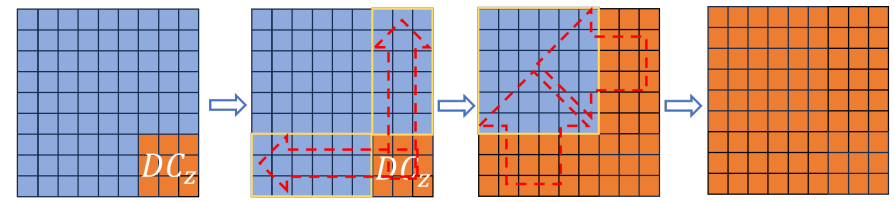


Fig. 3. The example that concretizes lines 3 -17 of Algorithm 7. (a) shows the concrete process of lines 3 - 10 of Algorithm 7, which updates DC_z. (b) presents lines 11 - 13 of Algorithm 7, which updates the submatrices in the horizontal and vertical directions. (c) shows the rest submatrices of DC_z in lines 14 - 17.

Lemma 4. *Given the adjacency matrix A of a weighted graph, the matrix DC computed by Algorithm 1. Algorithm 7 updates the matrix DC correctly under the $N(z)$ inserted edges ($z \leq n$).*

Proof. In Algorithm 7, we partition the matrix DC into submatrices of size $z \times z$. When $z \leq n^{\alpha/2}$, the diagonal block DC_z can be stored in a single machine. This means, when Algorithm 7 execute the last diagonal submatrix DC_z (refer to line 2 of Algorithm 1 and Fig. 3(a)), DC_z has already traversed all intermediate nodes with IDs in $\{1, 2, \cdots, n-z\}$. Since the distances between the z affected nodes only decrease under the $N(v)$ udpate edges, DC_z must be compared with DC'_z before processing the final z intermediate vertices (refer to Fig. 2(c)).

Then, Algorithm 1 and Algorithm 2 are applied to DC_z to update distances and shortest-path counts for these z nodes passing through the last z intermediate vertices. Subsequently, Algorithm 7 updates the vertical and horizontal submatrices as well as the remaining submatrices following lines 6 - 7 and line 12 of Algorithm 1. Hence, Algorithm 7 updates the distances and shortest-path counts between nodes correctly.

Lemma 5. *Given the adjacency matrix A of a weighted graph, the matrix DC computed by Algorithm 1 and $N(v)$ update edges. Algorithm 7 updates the matrix DC in $O(\alpha^{-1})$ rounds using $O(n^2)$ total memory when $z \leq n^{\alpha/2}$, and $O(z/(n^{\alpha/2}\alpha))$ rounds using $O(n^{3-\alpha/2})$ total memory for $z > n^{\alpha/2}$.*

Proof. In lines 2, 7, 9 - 10, 15, and 18 - 19 of Algorithm 7, the sorting and broadcast operations take $O(\alpha^{-1})$ rounds. When $z \leq n^{\alpha/2}$, Algorithm 7 updates DC following the framework of Algorithm 1 with $O(n^2)$ total memory. When $z > n^{\alpha/2}$, Algorithm 7 updates DC_z using Algorithm 1, which takes $O(z/(n^{\alpha/2}\alpha))$ rounds. In addition, lines 16 and 20 execute Algorithm 6, which take $O(\alpha^{-1})$ and $O(n^{3-\alpha/2})$ total memory.

5.3 The Parallel BC algorithm

As noted in Section 4.3, we replace the backward pass of SPBA by leveraging the path decomposition through vector operations. Concretely, we build upon an important observation from Brandes [5]: For any $u \in V$, $\sigma_{xy}(u) = 0$ if $d(x,y) < d(x,u) + d(u,y)$; $\sigma_{xy}(u) = \sigma_{xu} \cdot \sigma_{uy}$, otherwise, where $x, y \in V$.

In the MPC model, with DC computed by Algorithm 1 or updated Algorithms 4, 5, and 7, for each vertex $k' \in V = \{1, 2, \cdots, n\}$, we define:

$$C^{k'} = DC[:,k'] \cdot DC[k',:] = \left[(D[i',j'], \sigma_{i'k'} \cdot \sigma_{k'j'})\right]_{i',j' \in V}$$

Here, $DC[:,k'] = [(D[1,k'], \sigma_{1k'}),\ (D[2,k'], \sigma_{2k'})\ \cdots\ (D[n,k'], \sigma_{nk'})]^T$ and $DC[k',:] = [(D[k',1], \sigma_{k'1}),\ (D[k',2], \sigma_{k'2})\ \cdots\ (D[k',n], \sigma_{k'n})]$. For $i', j' \in V$, $\sigma_{i'j'}(k') = \sigma_{i'k'} \cdot \sigma_{k'j'}$, if $D[i',j'] = D[i',k'] + D[k',j']$; otherwise, $\sigma_{i'j'}(k') = 0$. Furthermore, we use $C^{k'}$ *div* DC to represent the matrix $BC^{k'}$ that is computed by element-wise division of $C^{k'}$ and Σ^0:

$$BC^{k'} = C^{k'}\ \ div\ \ DC = \left[\frac{\sigma_{i'j'}(k')}{\sigma_{i'j'}}\right]_{i',j' \in V}$$

$$BC^{k'}[i',j'] = \begin{cases} \frac{\sigma_{i'k'} \cdot \sigma_{k'j'}}{\sigma_{i'j'}} & \text{if } D[i',j'] = D[i',k'] + D[k',j'], \\ 0 & \text{otherwise.} \end{cases} \tag{3}$$

By summing all $BC^{k'}[i',j']$ for $i', j' \in V$, we obtain $C_B(k')$ (see Definition 1) for each k'. We provide Algorithm 8 that computes the BC values for all nodes in Appendix C. Specifically, lines 1 - 3 of Algorithm 8 broadcast required submatrices of DC among processors. In line 4, each processor computes an $n^{\frac{\alpha}{2}} \times n^{\frac{\alpha}{2}}$ submatrix $BC^{k'}_{i,j}$ for $k' \in V$ and sums its entries for a partial contribution of $C_B(k')$. Each processor handles $O(n^{\alpha/2})$ submatrices. Line 5 reduces to $C_B(k')$ for all k'. Lemma 6 (proved in Appendix C) gives the complexity of Algorithm 8. Appendix D analyzes the approximation ratio of the BC values.

Lemma 6. *For an n-vertex unweighted graph G and the matrix DC computed by Algorithm 1 or updated by Algorithms 4, 5, and 7, Algorithm 8 calculates the BC values for vertices in $O(1/\alpha)$ rounds ($\alpha \in (0,1)$) and $O(n^{3-\frac{\alpha}{2}})$ total memory.*

Remark: We can compute the stress centrality of all vertices using Algorithm 8, by modifying formula (3): $BC^{k'}[i', j'] = \sigma_{i'k'} \cdot \sigma_{k'j'}$, if $D[i', j'] = D[i', k'] + D[k', j']$; otherwise, $BC^{k'}[i', j'] = 0$, for all $i', j', k' \in V$.

6 Conclusion and Future Work

This paper studies the problem of incremental BC for weighted graph in the MPC model. Each processor has $O(n^\alpha)$ memory for a constant $\alpha \in (0, 1)$. The weighted graph contains n nodes and m edges. We utilize the blocked Floyd-Warshall algorithm and vector operations to replace the direct MPC implementation of the Brandes algorithm, thereby reducing the round complexity of preprocessing the initial graph from $O(n/\alpha)$ to $O(n^{1-\alpha/2}/\alpha)$. For three types of incremental updates (single-edge, single-vertex, and multiple-edge updates), we propose three parallel incremental BC update algorithms, all of which achieve $O(\alpha^{-1})$ rounds. However, for multiple-edge updates, when the number of involved vertices $z > n^{\alpha/2}$, the parallel incremental algorithm requires $O(z/(n^{\alpha/2}\alpha))$ rounds.

Technically, we leverage the Floyd-Warshall algorithm framework, combined with matrix multiplication and vector operations, to reduce the round complexity. However, these algebraic methods are applicable only to incremental update cases. For decremental updates, such as single-edge deletion or weight increase, the impact may propagate throughout the entire graph. Thus, designing an efficient decremental or fully dynamic BC algorithm remains a significant challenge.

References

1. Abboud, A., Grandoni, F., Vassilevska Williams, V.: Subcubic equivalences between graph centrality problems, APSP and diameter. In: Proceedings of the SODA. pp. 1681–1697. SIAM (2015)
2. Bader, D.A., Madduri, K.: Parallel algorithms for evaluating centrality indices in real-world networks. In: Proceedings of the ICPP. pp. 539–550. IEEE Computer Society (2006)
3. Balliu, A., Latypov, R., Maus, Y., Olivetti, D., Uitto, J.: Optimal deterministic massively parallel connectivity on forests. In: Proceedings of the SODA. pp. 2589–2631. SIAM (2023)
4. Boyd, S., Vandenberghe, L.: Convex Optimization. Cambridge University Press, Cambridge, UK (2004)
5. Brandes, U.: A faster algorithm for betweenness centrality. The Journal of Mathematical Sociology **25**(2), 163–177 (2001)
6. Demetrescu, C., Italiano, G.F.: A new approach to dynamic all pairs shortest paths. J. ACM **51**(6), 968–992 (2004)
7. Dhulipala, L., Durfee, D., Kulkarni, J., Peng, R., Sawlani, S., Sun, X.: Parallel batch-dynamic graphs: Algorithms and lower bounds. In: Proceedings of the SODA. pp. 1300–1319 (2020)
8. Dinitz, M., Nazari, Y.: Massively parallel approximate distance sketches. In: Proceedings of the OPODIS. pp. 35:1–35:17. Schloss Dagstuhl – Leibniz-Zentrum für Informatik (2019)

9. Ghaffari, M., Nowicki, K.: Massively parallel algorithms for minimum cut. In: Proceedings of the PODC. pp. 119–128. ACM (2020)
10. Gilbert, S., Lu, L.L.E.: How fast can you update your mst? In: Proceedings of the SPAA. pp. 531–533. ACM (2020)
11. Goel, K., Singh, R.R., Gupta, S., Iyengar, S.R.S.: A faster algorithm to update betweenness centrality after node alteration. Internet Math. **11**(4-5), 403–420 (2015)
12. Goodrich, M.T., Sitchinava, N., Zhang, Q.: Sorting, searching, and simulation in the mapreduce framework. In: Proceedings of the ISAAC. pp. 374–383. Springer (2011)
13. Green, O., McColl, R., Bader, D.A.: A fast algorithm for streaming betweenness centrality. In: Proceedings of the PASSAT. pp. 11–20. IEEE Computer Society (2012)
14. Hajiaghayi, M., Lattanzi, S., Seddighin, S., Stein, C.: Mapreduce meets fine-grained complexity: Mapreduce algorithms for apsp, matrix multiplication, 3-sum, and beyond. CoRR **abs/1905.01748** (2019)
15. Hua, Q., Fan, H., Ai, M., Qian, L., Li, Y., Shi, X., Jin, H.: Nearly optimal distributed algorithm for computing betweenness centrality. In: Proceedings of the ICDCS. pp. 271–280. IEEE (2016)
16. Italiano, G.F., Lattanzi, S., Mirrokni, V.S., Parotsidis, N.: Dynamic algorithms for the massively parallel computation model. In: Proceedings of the SPAA. pp. 49–58. ACM (2019)
17. Jamour, F.T., Skiadopoulos, S., Kalnis, P.: Parallel algorithm for incremental betweenness centrality on large graphs. IEEE Trans. Parallel Distributed Syst. **29**(3), 659–672 (2018)
18. Kas, M., Wachs, M., Carley, K.M., Carley, L.R.: Incremental algorithm for updating betweenness centrality in dynamically growing networks. In: Proceedings of the ASONAM. pp. 33–40. ACM (2013)
19. Lee, M., Lee, J., Park, J.Y., Choi, R.H., Chung, C.: QUBE: a quick algorithm for updating betweenness centrality. In: Proceedings of the 21st World Wide Web Conference. pp. 351–360. ACM (2012)
20. Nasre, M., Pontecorvi, M., Ramachandran, V.: Betweenness centrality - incremental and faster. In: Proceedings of the MFCS. Lecture Notes in Computer Science, vol. 8635, pp. 577–588. Springer (2014)
21. Nasre, M., Pontecorvi, M., Ramachandran, V.: Decremental all-pairs ALL shortest paths and betweenness centrality. In: Proceedings of the ISAAC. Lecture Notes in Computer Science, vol. 8889, pp. 766–778. Springer (2014)
22. Nowicki, K., Onak, K.: Dynamic graph algorithms with batch updates in the massively parallel computation model. In: Proceedings of the SODA. pp. 2939–2958. SIAM (2021)
23. Pontecorvi, M., Ramachandran, V.: Fully dynamic betweenness centrality. In: Proceedings of the ISAAC. Lecture Notes in Computer Science, vol. 9472, pp. 331–342. Springer (2015)
24. Ramalingam, G., Reps, T.W.: On the computational complexity of dynamic graph problems. Theor. Comput. Sci. **158**(1&2), 233–277 (1996)
25. Sao, P., Kannan, R., Gera, P., Vuduc, R.W.: A supernodal all-pairs shortest path algorithm. In: Proceedings of the PPoPP. pp. 250–261. ACM (2020)
26. Shukla, K., Regunta, S.C., Tondomker, S.H., Kothapalli, K.: Efficient parallel algorithms for betweenness- and closeness-centrality in dynamic graphs. In: Proceedings of the ICS. pp. 10:1–10:12. ACM (2020)
27. Tan, G., Tu, D., Sun, N.: A parallel algorithm for computing betweenness centrality. In: Proceedings of the ICPP. pp. 340–347. IEEE (2009)

28. Wang, C., Hua, Q., Jin, H., Zheng, C.: Massively parallel algorithms for fully dynamic all-pairs shortest paths. Frontiers Comput. Sci. **18**(4), 184611 (2024)

A The Pseudocode and Detailed Analysis of SPBA

To parallelize the Brandes algorithm in the MPC model, we assume that the vertices in V are assigned unique IDs from 1 to n. The edges incident to each vertex are arranged on a consecutive set of processors and sorted by the ID of the neighboring endpoint. Accordingly, a vertex and its related edges with smaller IDs are stored in processors with smaller IDs [8].

The pseudocode of the parallel Brandes algorithm for weighted graphs is given in Algorithm 3. First, line 2 initializes σ_{xy}, $d(x, y)$, and $h_x(y)$ for all $x, y \in V$, where $h_x(y)$ records the maximum number of edges on the shortest path from x to y.

In the forward pass, lines 3 - 22 compute distances and shortest-path counts between nodes. In the i-th iteration, for each node u, only predecessors v of u that satisfy $h_x(v) = i - 1$ participate in the computation (refer to lines 8 - 20 of Algorithm 3). Specifically, if these predecessors v with $h_x(v) = i - 1$ improve $d(x, u)$ (line 10), then the predecessor set $P_x(u)$ is reset to empty (line 11) and $h_x(u) = i$(line 12). If these predecessors v with $h_x(v) = i - 1$ are newly added predecessors (refer to line 14), then only $h_x(u)$ is set to i (line 15). Afterwards, lines 17 - 19 compute $\sigma_x(u)$ and the predecessor set $P_x(u)$ for u, and broadcast these results to the processors storing the outgoing edges of u.

The forward pass runs for n rounds. This is because any simple shortest path has at most $n-1$ edges in a weighted graph. Moreover, n iterations are sufficient to guarantee convergence of all distances and shortest-path counts for all pairs of nodes.

In the backward pass, lines 23 - 32 compute the dependency values following the Brandes algorithm framework. The backward pass proceeds in decreasing order of the maximum number of edges on the shortest path (refer to line 23). In each round, only nodes v with $h_x(v) = i$ are processed (line 24). The processors storing these nodes broadcast their dependency values $\delta_x(v)$ and shortest-path counts σ_{xv} to the processors that store the predecessors $u \in P_x(v)$(line 25). After receiving these messages, each processor computes local contribution $\delta'_x(u)$ (refer to line 26) and then aggregates them via reduction operation to obtain the final $\delta_x(u)$(lines 27 - 28). Finally, all nodes $u \neq x$ accumulate the dependency values into their partial BC value $C_B(u)$ (lines 29 - 31). The backward pass also requires n iterations.

Finally, in lines 33 -35, we compute the final BC values for all nodes in parallel.

For the large value problem, we adopt the floating-point arithmetic technique proposed in [15] and omit further details here.

B The pseudocodes of Algorithms 4, 5, 6, and 7

We present Algorithms 4, 5, 6, and 7 as follows:

Algorithm 3 The Straightforward Parallel Brandes Algorithm

Input: A weighted graph $G = (V, E)$, which is replicated for n times, each copy is distributed among a continuous set of processors. For each continuous set of processors, set a different node as the source node.

Output: $C_B(u)$, where $u \in V$.

1: **for** all source $x \in V$ in parallel **do**
2: The processor that stores x, sets $\sigma_{xx} = 1, d(x,x) = 0, h_x(x) = 0, \delta_{x\cdot}(x) = 0$ and $C_B(x) = 0$. The processors that store $y \in V \setminus \{x\}$, initialize $P_x(y) = \emptyset$, $\sigma_{xy} = 0$, $d(x,y) = +\infty$, $h_x(y) = +\infty$, $\delta_{x\cdot}(y) = 0$, and $C_B(y) = 0$.
3: **for** $i = 1$ to n **do** ▷ Forward pass
4: **for** $u \in V$ **do**
5: **if** $u == x$ **then**
6: The processor that stores x, broadcasts $d(x,x)$, σ_{xx}, and $h_x(x)$ to the processors that store the outgoing neighbors of x.
7: **end if**
8: **if** $u \neq x$ **then**
9: The processors that store u and its edges, compute $d^{new}(x,u) = \min_{\substack{v \in N_{in}(u) \\ h_x(v) = i-1}} \{d(x,v) + w(v,u)\}$ ▷ $N(u)$ is the incoming neighbors set of u and the processors that store u and its edges, have received $d(x,v)$ and σ_{xv} with $(v,u) \in N_{in}(u)$ from other processors.
10: **if** $d^{new}(x,u) < d(x,u)$ **then**
11: set $P_x(u) = \emptyset$ and $d(x,u) = d^{new}(x,u)$
12: The processor that stores u sets $h_x(u) = i$
13: **end if**
14: **if** $d^{new}(x,u) = d(x,u)$ **and** $d(x,u) < +\infty$ **then**
15: The processor that stores u sets $h_x(u) = i$ ▷ Update the max number of edges on the shortest path from x to u
16: **end if**
17: The processors that store the edges (v,u) with $d(x,u) = d(x,v) + w(v,u)$, append $v \rightarrow P_x(u)$ and compute a local partial sum $\sigma'_{xu} = \sum_{\substack{v \in N_{in}(u) \\ d(x,u) = d(x,v) + w(v,u)}} \sigma_{xv}$ ▷ Each processor stores at most $O(n^\alpha)$ edges of u and there may be more than one neighbor v of u with $d(x,u) = d(x,v) + 1$.
18: The processor that stores u, aggregates all local σ'_{xu} via a reduction operation to obtain the final σ_{xu}
19: The processor that stores u, broadcasts $d(x,u)$, σ_{xu}, and $h_x(u)$ to the processors that store u's outgoing edges.
20: **end if**
21: **end for**
22: **end for**
23: **for** $i = n$ to 1 **do** ▷ The backward pass
24: **for** $v \in V$ with $h_x(v) = i$ **do**
25: The processors that store v, broadcast δ_{xv} and σ_{xv} to the processors that store u with $u \in P_x(v)$
26: The processors that store the edges (u,v) with $u \in P_x(v)$, compute $\delta'_{x\cdot}(u) = \sum_{v:u \in P_x(v)} \frac{\sigma_{xu}}{\sigma_{xv}}(1 + \delta_{x\cdot}(v))$
27: The processor that stores u, receives these $\delta'_{x\cdot}(u)$ from the processors that store u's edges by a reduction operation
28: The processor storing u, sums these $\delta'_{x\cdot}(u)$ and gets $\delta_{x\cdot}(u) += \delta'_{x\cdot}(u)$
29: **if** $u \neq x$ **then**
30: The processors storing $u \in P_x(v)$, compute $C_B(u) = C_B(u) + \delta_{x\cdot}(u)$
31: **end if**
32: **end for**
33: **end for**
34: **for** all nodes $u \in V$ in parallel **do**
35: All processors that store node u and its partial BC values aggregate these partial BC values via a reduction operation to obtain the final $C_B(u)$
36: **end for**
37: **end for**

Algorithm 4 The Single-edge Update Algorithm for BC

Input: The $n \times n$ matrix DC with n^2 tuples (computed by Algorithm 1) and an $n \times n$ empty matrix DC' with n^2 tuples, which are distributed among $O(n^{2-\alpha})$ machines numbered as $P_{i,j}, i, j \in \{1, 2, \cdots, n^{1-\alpha/2}\}$. An updated edge (v, u) with its weight $w(v, u)$.

Output: The updated matrix DC.

1: Broadcast $w(v, u)$ to the processors P_{i,j_v} that store the sub-vectors $DC[(i-1) \cdot n^{\alpha/2} + 1 : i \cdot n^{\alpha/2}, v]$, $j \neq j_v$ and $i \in \{1, 2, \cdots, n^{1-\alpha/2}\}$. ▷ j_v represents the column index of processors that store the distances and shortest path counts from all nodes to u

2: The processors P_{i,j_v} that store the sub-vectors $DC[(i-1) \cdot n^{\alpha/2} + 1 : i \cdot n^{\alpha/2}, v]$, compute $DC'[i', u] = (D'[i', v], \sigma'_{i',u}) = (D[i', v] + w(v, u), \sigma_{i',v})$ ▷ $\sigma_{i',u} = \sigma_{i',v}$ since only the edge (v, u) is considered

3: The processors P_{i,j_v} broadcast the sub-vectors $DC'[(i-1) \cdot n^{\alpha/2} + 1 : i \cdot n^{\alpha/2}, v]$ broadcasts to other $O(n^{1-\alpha/2})$ machines numbered by $P_{i,j}, j \neq j_v$ and $j \in \{1, 2, \cdots, n^{1-\alpha/2}\}$.

4: The processors $P_{i_u,j}$ broadcast the sub-vectors $DC[u, (j-1) \cdot n^{\alpha/2} + 1 : j \cdot n^{\alpha/2}]$ broadcasts to other $O(n^{1-\alpha/2})$ machines numbered by $P_{i,j}, i \neq i_u$ and $i \in \{1, 2, \cdots, n^{1-\alpha/2}\}$. ▷ i_u represents the row index of processors that store the distances and shortest path counts from v to all nodes

5: Each $P_{i,j}$ computes the outer product of two sub-vectors $DC'[(i-1) \cdot n^{\alpha/2} + 1 : i \cdot n^{\alpha/2}, u]$ and $DC[u, (j-1) \cdot n^{\alpha/2} + 1 : j \cdot n^{\alpha/2}]$ with $DC'[i', j'] = (D'[i', j'], \sigma'_{i',j'}) = (D'[i', u] + D[u, j'], \sigma'_{i',u} \cdot \sigma_{u,j'})$

6: Each processor $P_{i,j}$ updates the submatrix DC_{ij} with the submatrix DC'_{ij} using formula (2).

7: Return DC.

Algorithm 5 The Single-vertex Update Algorithm for BC

Input: The $n \times n$ matrix DC with n^2 tuples (computed by Algorithm 1) and an $n \times n$ empty matrix DC' with n^2 tuples, which are distributed among $O(n^{2-\alpha})$ machines numbered as $P_{i,j}, i, j \in \{1, 2, \cdots, n^{1-\alpha/2}\}$. The vector VC^{in} and VC^{out} are distributed among $O(n^{1-\alpha/2})$ processors.

Output: The updated matrix DC.

1: The processors P_{i,j_v} broadcast the sub-vector VC_i^{in} to the processors $P_{j,i}$ that store the sub-matrix DC_{ji} with size $n^{\alpha/2} \times n^{\alpha/2}$, where $j \neq j_v$ and $j \in \{1, 2, \cdots, n^{1-\alpha/2}\}$. ▷ j_v represents the column index of processors that store the distances and shortest path counts from all nodes to v and $VC_i^{in} = VC^{in}[(i-1) \cdot n^{\alpha/2} + 1 : i \cdot n^{\alpha/2}]$

2: Each processor $P_{j,i}$ computes the multiplication of DC_{ji} and VC_i^{in} using formula

3: The processors $P_{j,i}$ compute the partial sums for the final VC_i^{in} by reduction operations

4: The processors $P_{i_v,j}$ broadcast the sub-vector VC_j^{out} to the processors $P_{j,i}$ that store DC_{ji} with size $n^{\alpha/2} \times n^{\alpha/2}$, where $i \neq i_v$ and $i \in \{1, 2, \cdots, n^{1-\alpha/2}\}$. ▷ i_v represents the row index of processors that store the distances and shortest path counts from v to all nodes and $VC_j^{out} = VC^{out}[(j-1) \cdot n^{\alpha/2} + 1 : j \cdot n^{\alpha/2}]$

5: Each processor $P_{j,i}$ computes the multiplication of VC_j^{out} and DC_{ji}

6: The processors $P_{j,i}$ compute the partial sums for the final VC_i^{out} by reduction operations

7: The processors P_{i,j_v} broadcast VC_i^{in} to other $O(n^{1-\alpha/2})$ machines numbered by $P_{i,j}, j \neq j_v$ and $j \in \{1, 2, \cdots, n^{1-\alpha/2}\}$.

8: The processors $P_{i_v,j}$ broadcast VC_j^{out} to other $O(n^{1-\alpha/2})$ machines numbered by $P_{i,j}, i \neq i_v$ and $i \in \{1, 2, \cdots, n^{1-\alpha/2}\}$.

9: Each $P_{i,j}$ computes the outer product of two sub-vectors VC_i^{in} and VC_j^{out} with $DC'[i', j'] = (D'[i', j'], \sigma'_{i',j'}) = (V^{in}[i'] + V^{out}[j'], \sigma'_{i'v} \cdot \sigma_{vj'})$

10: Each processor $P_{i,j}$ updates the submatrix DC_{ij} with the submatrix DC'_{ij} using formula (2).

11: Return $DC = \begin{bmatrix} DC & VC^{in} \\ VC^{out} & (0,1) \end{bmatrix}$. ▷ The tuple $(0,1)$ means $d(v,v) = 0$ and $\sigma_{vv} = 1$

Algorithm 6 ParMMDC(DC^1, DC^2, DC^3, n)

Input: Three $n \times n$ matrices DC^1, DC^2, and DC^3, each has n^2 tuples, which are distributed across $O(n^{2-\alpha})$ processors $P_{i,k,1}$ $(i, k \in \{1, 2, \cdots, n^{1-\frac{\alpha}{2}}\})$, where $DC^i = \left[D_i[i', j'], \sigma^i_{i'j'}\right]_{i',j' \in \{1,2,\cdots,n\}}$

Output: The solution matrix $DC^3 = DC^1 \times DC^2$

1: The processor $P_{i,k,1}$ broadcasts the $n^{\frac{\alpha}{2}} \times n^{\frac{\alpha}{2}}$ submatrix DC^1_{ik} to the other $O(n^{1-\alpha/2})$ processors $P_{i,k,j}$ $(j \in \{2, 3, \cdots, n^{1-\frac{\alpha}{2}}\})$

2: The processor $P_{i,k,1}$ broadcasts the submatrix DC^2_{ik} to the other $O(n^{1-\frac{\alpha}{2}})$ processors $P_{j,i,k}$ $(j \in \{1, 2, \cdots, n^{1-\frac{\alpha}{2}}\})$

3: Each processor $P_{i,k,j}$ executes SequFW(DC^1_{ik},DC^2_{kj},DC^3_{ij},$Dist$,$n^{\alpha/2}$), where $i, k, j \in \{1, 2, \cdots, n^{1-\frac{\alpha}{2}}\}$

4: The processors $P_{i,k,j}$ calculate the partial sums of DC^3_{ij} and get the final DC^3_{ij} by reduction operations

5: Return DC^3

Algorithm 7 The Multiple-edge Update Algorithm for BC

Input: The $n \times n$ adjacency matrix A of the given weighted graph, the matrix DC computed by Algorithm 1, and an $n \times n$ empty matrix DC' with n^2 tuples, which are distributed among $O(n^{2-\alpha})$ processors $P_{i,j}$, where $i, j \in \{1, 2, \cdots, n^{1-\alpha/2}\}$. The weights of $N(z)$ updated edges.

Output: The updated DC.

1: Update the adjacency matrix A with the $N(z)$ updated edges
2: Reorder the vertices in matrix A so that the edge weights between the affected z vertices are rearranged in the last z rows and last z columns of matrix A. Perform the same operation on DC
3: Use DC'_z of size $z \times z$ to store the weights and shortest-path counts of these z nodes, and set $DC'[i', j'] = (D'[i', j'], \sigma_{i'j'})$ with $D'[i', j'] = A[i', j']$ and $\sigma_{i'j'} = 0$ for $i' \neq j'$ and $\sigma_{i'j'} = 1$ for $i' = j'$, where $i', j' \in \{n - z + 1, n - z + 2, \cdots, n\}$
4: The processors $P_{i,j}$updates the submatrix DC_z with the submatrix DC'_z using formula (2), where $i, j \in \{(n - z)/n^{\alpha/2}, n^{1-\alpha/2}\}$ ▷ DC_z with $z \times z$ stores the distances and shortest-path counts between these z affected nodes
5: **if** $z \leq n^{\alpha/2}$ **then**
6: The processor $P_{n^{1-\alpha/2},n^{1-\alpha/2}}$ executes SequFw($DC_{q,q}$,$DC_{q,q}$,$DC_{q,q}$,$Dist_{q,q}$,$n^{\alpha/2}$), where $q = n^{1-\alpha/2}$ and $Dist_{q,q}$ is an $n^{\alpha/2} \times n^{\alpha/2}$ empty matrix ▷ Update the distances and shortest-path counts among the last $n^{\alpha/2}$ nodes instead of the z nodes, since DC_z can be stored on a single processor
7: The processors that store $DC_{n^{1-\alpha/2},n^{1-\alpha/2}}$ broadcast it to the processors storing the submatrices $DC_{i,n^{1-\alpha/2}}$ and $DC_{n^{1-\alpha/2},j}$ with size $z \times z$, where $i, j \in \{1, 2, \cdots, n^{1-\alpha/2}\}$
8: The processors that store $DC_{i,q}$ (or $DC_{q,j}$) execute SequFw($DC_{i,q}$,$DC_{q,q}$, $DC_{i,q}$, $Dist_{i,q}$,$n^{\alpha/2}$) (or SequFw($DC_{q,q}$, $DC_{q,j}$,$DC_{q,j}$,$Dist_{q,j}$,$n^{\alpha/2}$), where $Dist_{i,q}$ (or $Dist_{q,j}$) is an $n^{\alpha/2} \times n^{\alpha/2}$ empty matrix ▷ Update the $n^{1-\alpha/2}$-th row or the $n^{1-\alpha/2}$-th column submatrices
9: The processors that store $DC_{i,n^{1-\alpha/2}}$ broadcast it to the processors storing $DC_{i,j}$ with size $n^{\alpha/2} \times n^{\alpha/2}$, where $i, j \in \{1, 2, \cdots, n^{1-\alpha/2}\}$
10: The processors that store $DC_{n^{1-\alpha/2},l}$ broadcast it to the processors storing $DC_{i,j}$ with size $n^{\alpha/2} \times n^{\alpha/2}$, where $i, j \in \{1, 2, \cdots, n^{1-\alpha/2}\}$
11: The processors that store $DC_{k,l}$ execute SequFw($DC_{k,n^{1-\alpha/2}}$,$DC_{n^{1-\alpha/2},l}$, $DC_{k,l}$, $Dist_{k,l}$,$n^{\alpha/2}$) ▷ Update the rest submatrices
12: **end if**
13: **if** $z > n^{\alpha/2}$ **then**
14: The processors $P_{i,j}$ update the $z \times z$ submatrix DC_z using Algorithm 1, where $i, j \in \{(n - z)/n^{\alpha/2}, n^{1-\alpha/2}\}$
15: The processors that store DC_z broadcast it to the processors storing the submatrices $DC_{k,n/z}$ and $DC_{n/z,k}$ with size $z \times z$, where $k \in \{1, 2, \cdots, n/z\}$
16: The processors that store $DC_{k,n/z}$ (or $DC_{n/z,k}$) execute ParMMDC($DC_{k,n/z}$, DC_z, $DC'_{k,n/z}$, z) (or ParMMDC(DC_z, $DC_{n/z,k}$, $DC'_{n/z,k}$, z)) ▷ Compute the product $DC'_{k,n/z}$ (or $DC'_{n/z,k}$) of two matrices $DC_{k,n/z}$ and DC_z (DC_z and $DC_{n/z,k}$) and refer to Algorithm 6
17: The processors that store $DC_{k,n/z}$ (or $DC_{n/z,k}$) update it with $DC'_{k,n/z}$ (or $DC'_{n/z,k}$), using formula (2)
18: The processors that store $DC_{k,n/z}$ broadcast it to the processors storing $DC_{k,l}$ with size $z \times z$, where $k, l \in \{1, 2, \cdots, n/z\}$
19: The processors that store $DC_{n/z,l}$ broadcast it to the processors storing $DC_{k,l}$ with size $z \times z$, where $k, l \in \{1, 2, \cdots, n/z\}$
20: The processors that store $DC_{k,l}$ execute ParMMDC($DC_{k,n/z}$, $DC_{n/z,l}$, $DC'_{k,l}$, z) ▷ Compute the product $DC'_{k,l}$ of $DC_{k,n/z}$ and $DC_{n/z,l}$
21: The processors that store $DC_{k,l}$ update it with $DC'_{k,l}$, using formula (2)
22: **end if**
23: **return** DC

C The pseudocode of Algorithm 8 and the proof of Lemma 6

We present Algorithm 8 below: Now, we prove Lemma 6 of Section 5.3 below:

Algorithm 8 The Parallel Betweenness Centrality Algorithm

Input: The $n \times n$ matrix DC with n^2 tuples computed by Algorithm 1, is distributed across $O(n^{2-\alpha})$ processors $P_{i,k,1}$ $(i,k \in \{1,2,\cdots,n^{1-\frac{\alpha}{2}}\})$.

Output: The BC vector $C_B = [C_B(1)\ C_B(2)\ \cdots\ C_B(n)]^T$ distributed across processors $P_{i,1,1}$, $i \in \{1,2,\cdots,n^{1-\frac{\alpha}{2}}\}$.

1: The processor $P_{i,k,1}$ broadcasts the $n^{\frac{\alpha}{2}} \times n^{\frac{\alpha}{2}}$ submatrix DC_{ik} to the other $O(n^{1-\frac{\alpha}{2}})$ processors $P_{i,k,j}$ $(i,k,j \in \{1,2,\cdots,n^{1-\frac{\alpha}{2}}\})$.

2: The processor $P_{i,k,1}$ broadcasts the $n^{\frac{\alpha}{2}} \times n^{\frac{\alpha}{2}}$ submatrix DC_{ik} to the other $O(n^{1-\frac{\alpha}{2}})$ processors $P_{i,j,k}$.

3: The processor $P_{i,k,1}$ broadcasts the submatrix DC_{ik} with size $n^{\frac{\alpha}{2}} \times n^{\frac{\alpha}{2}}$ to the other $O(n^{1-\frac{\alpha}{2}})$ processors $P_{j,i,k}$ $(i,k(\neq 1) \in \{1,2,\cdots,n^{1-\frac{\alpha}{2}}\})$.

4: Each processor $P_{i,k,j}$ $(i,k,j \in \{1,2,\cdots,n^{1-\frac{\alpha}{2}}\})$ computes $BC_{ij}^{k'}$ with size $n^{\frac{\alpha}{2}} \times n^{\frac{\alpha}{2}}$ for each $k' \in \{(k-1)\cdot n^{\frac{\alpha}{2}}+1,\cdots,k\cdot n^{\frac{\alpha}{2}}\}$ and sums all elements in $BC_{i,j}^{k'}$ to get the partial BC value for the vertex k', using the formula (3), $DC_{ik}[:,k']$, DC_{ij}, and $DC_{jk}[k',:]$. ▷ Each processor accumulates at most $O(n^{\alpha})$ values of partial BC for at most $O(n^{\frac{\alpha}{2}})$ vertices, since there are n^2 pairs of vertices.

5: The reduction operation is performed among processors $P_{i,k,j}$ $(i,k,j \in \{1,2,\cdots,n^{1-\frac{\alpha}{2}}\})$ to calculate the partial sums of $C_B(k')$ and obtain the final $C_B(k')$ for all $k' \in V$.

Proof (Lemma 6). As described above, in Algorithm 8, only the broadcast operations in lines 1 - 3 and the reduction operation in line 5 are performed among processors, each requiring $O(1/\alpha)$ round complexity. Moreover, Algorithm 8 utilizes $O(n^{3-\frac{3\alpha}{2}})$ processors, each with $O(n^{\alpha})$ memory size, resulting in $O(n^{3-\frac{\alpha}{2}})$ total memory.

D Large-Value Computation: Logarithmic Representation and Error Analysis

As mentioned in Section 4.3, we represent σ_{xy} in logarithmic form as $l_{xy} = \log_2(\sigma_{xy})$ to solve the large-value problem. For $x,v,u \in V$, we define $l_{xv} = \log_2(\sigma_{xv}), l_{xv} = \log_2(\sigma_{xv})$, and $l_{vu} = \log_2(\sigma_{vu})$ and compute l_{xv} using the formula (4) as follows:

$$l_{xu} = \log_2\Big(\sum_{\substack{v \in V_h \\ D[x,u]=D[x,v]+D[v,u]}} 2^{l_{xv}+l_{vu}}\Big) \tag{4}$$

Next, we show that, by choosing sufficiently high storage precision, the final betweenness centrality (BC) values computed in Section 5.3 achieve a multiplicative $(1 \pm \varepsilon)$-approximation. This approximation does not affect the round complexity or total memory of our parallel algebraic algorithms.

Fixed-Point Logarithmic Storage We store each l_{xy} as a fixed-point number:

• Integer part: Since each $l_{xy} \in [0, n \log n]$, its integer part requires only $\lceil \log_2(n \log n) \rceil = O(\log n)$ bits to cover this range.

• Fractional part: This part controls the absolute error. Let l_{xy} be the exact value and $\tilde{l}_{xy}$ its stored approximation. We require an absolute error bound Δ_0, i.e., $|\tilde{l}_{xy} - l_{xy}| \leq \Delta_0$. To achieve this precision, we need $2^{-F} \leq \Delta_0$, which gives $F \geq \lceil \log_2(1/\Delta_0) \rceil$ bits. Hence, storing each l_{xy} with F fractional bits guarantees an absolute error at most Δ_0. The exact value of F will be determined later.

In addition, for the distance $D[x, y] = \infty$, we set $l_{xy} = -\infty$ and store it as a special flag.

Lipschitz Property and Error Propagation of Algorithm 1 The formula (4) exhibits a typical log-sum-exp structure. To analyze the error propagation of this formula during computation, we define the function

$$f(a_1, \ldots, a_k) = \log_2 \left(\sum_{i=1}^{k} 2^{a_i} \right), \quad a_i \in \mathbb{R}.$$

Lemma 7 (Lipschitz Property). *For any vectors $\boldsymbol{a} = (a_1, \ldots, a_k)$ and $\boldsymbol{\delta} = (\delta_1, \ldots, \delta_k)$ with $a_i, \delta_i \in \mathbb{R}$ and $i \in \{1, \cdots, k\}$, if $|f(\boldsymbol{a}+\boldsymbol{\delta}) - f(\boldsymbol{a})| \leq \max_{1 \leq i \leq k} |\delta_i|$, then f is 1-Lipschitz with respect to the ℓ_∞ norm, where $\|\boldsymbol{a}\|_\infty = \max_i |a_i|$.*

Proof. The partial derivatives of f are

$$\frac{\partial f}{\partial a_i} = \frac{2^{a_i}}{\sum_{j=1}^{k} 2^{a_j}},$$

with $0 \leq \frac{\partial f}{\partial a_i} \leq 1$ and $\sum_{i=1}^{k} \frac{\partial f}{\partial a_i} = 1$. By the first-order Taylor expansion, for some $\xi \in (0, 1)$,

$$f(\boldsymbol{a} + \boldsymbol{\delta}) - f(\boldsymbol{a}) = \sum_{i=1}^{k} \frac{\partial f}{\partial a_i}(\boldsymbol{a} + \xi \boldsymbol{\delta}) \cdot \delta_i.$$

Hence,

$$|f(\boldsymbol{a} + \boldsymbol{\delta}) - f(\boldsymbol{a})| \leq \sum_{i=1}^{k} \frac{\partial f}{\partial a_i}(\boldsymbol{a} + \xi \boldsymbol{\delta}) \cdot |\delta_i| \leq \max_i |\delta_i| \cdot \sum_{i=1}^{k} \frac{\partial f}{\partial a_i} = \max_i |\delta_i|.$$

Thus, the output error is bounded by the maximum absolute input error and independent of the value of k. For further properties of the log-sum-exp function and its role in algorithm analysis, see [4].

Both the sequential Floyd-Warshall algorithm and Algorithm 1 traverse all intermediate vertices in a fixed order, namely, $k = 1, 2, \cdots, n$. In Algorithm 1, the i-th iteration processes $O(n^{\alpha/2})$ intermediate vertices simultaneously. Let Δ_{xv}^{i-1} and Δ_{vu}^{i-1} denote the accumulated errors of l_{xv} and l_{vu}. In the i-th iteration, if $D[x,u] = D[x,v] + D[v,u]$, then the error of $l_{xv} + l_{vu}$ is $\Delta_{xv}^{i-1} + \Delta_{vu}^{i-1}$. Since each iteration traverses at most $O(n^{\alpha/2})$ new intermediate nodes, any shortest path between any pair of nodes contains at most $i \cdot n^{\alpha/2}$ intermediate vertices. Hence, $\Delta_{xv}^{i-1} + \Delta_{vu}^{i-1} \leq i \cdot n^{\alpha/2} \cdot \Delta$ for a single intermediate node v, where Δ is the initial error. Since each iteration processes at most $O(n^{\alpha/2})$, by Lemma 7 and formula (4), $\Delta_{xu}^{i} \leq i \cdot n^{\alpha/2} \cdot \Delta$. Therefore, the final error Δ_{xu}^{T} for l_{xu} is at most $n\Delta_0$, where $T = n^{1-\alpha/2}$.

We set $\Delta_0 = \epsilon/n$, then final error $\Delta^T \leq \epsilon$. The required fractional bits are $F = \lceil \log_2(1/\Delta_0) \rceil = O(\log n + \log(1/\varepsilon))$. Therefore, using $O(\log n)$-bit fixed-point numbers ensures that, after Algorithm 1, every l_{xu} has absolute error at most ε. The total storage for each l_{xy} is $O(\log n) + O(\log n + \log(1/\varepsilon)) = O(\log n + \log(1/\varepsilon))$. In the MPC model, each word has $O(\log n)$ bits. Consequently, when ε is a constant or a polynomial in $1/n$, the memory overhead for each l_{xy} remains $O(\log n)$ bits, which occupies a constant number of words. This directly addresses the large-value problem by reducing the storage for the shortest-path count of each vertex pair from $O(d \log n)$ to $O(\log n)$ bits, thereby significantly cutting memory and communication costs.

Approximation Ratio of BC Values in Section 5.3 In this part, we analyze the approximation ratio of using fixed-point logarithmic representation.

Theorem 2. *In the MPC model, using fixed-point logarithmic representation (integer part $O(\log n)$ bits, fractional part $O(\log n + \log(1/\varepsilon))$ bits), Algorithm 8 computes the BC values $\tilde{C}_B(v)$ such that for all $v \in V$,*

$$(1-\varepsilon)C_B(v) \leq \tilde{C}_B(v) \leq (1+\varepsilon)C_B(v),$$

the $\log n$ factor remains when ε is constant.

Proof. As noted above, choosing $\Delta_0 = \varepsilon/n$ guarantees that after Algorithm 1 in Section 5.1, each l_{xy} has absolute error $\leq \varepsilon$.

In the actual computation of Algorithm 8, to avoid intermediate overflow from directly computing $2^{l_{xv}+l_{vu}}$, we employ a numerically stable log-domain addition:

$$l_{xu} = \max_v(l_{xv} + l_{vu}) \; + \; \log_2\Big(1 + \sum_{v \neq \arg\max} 2^{(l_{xv}+l_{vu}) - \max_v(l_{xv}+l_{vu})}\Big).$$

Since $l_{xv} + l_{vu} \leq \max_v(l_{xv} + l_{vu})$, the exponent $e = (l_{xv} + l_{vu}) - \max_v(l_{xv} + l_{vu})$ is non-positive. Consequently, 2^e lies in $(0, 1]$ and requires only $O(\log n + \log(1/\varepsilon))$ bits to store, avoiding the overflow problem caused by huge intermediate values (such as $2^{n \log n}$). The additional rounding error introduced by this stable computation is negligible. It can be fully compensated by reserving one extra

bit (one binary fractional bit) in the fractional part, without affecting the final absolute error bound ε.

In Algorithm 1, we have computed the matrix DC where $DC[i,j] = (D[i,j], \sigma_{ij})$. Here, we replace σ_{ij} with $l_{ij} = \log_2 \sigma_{ij}$. Recall from Algorithm **??** that for each vertex k, the contribution of a pair (i,j) is

$$BC^k[i,j] = \begin{cases} 2^{\,l_{ik}+l_{kj}-l_{ij}}, & \text{if } D[i,j] = D[i,k] + D[k,j], \\ 0, & \text{otherwise.} \end{cases} \tag{5}$$

Then $C_B(k) = \sum_{i,j} BC^k[i,j]$. Each term $r = 2^{l_{ik}+l_{kj}-l_{ij}}$ (when the distance condition holds) satisfies $0 < r \leq 1$. The estimated term $\tilde{r} = 2^{\tilde{l}_{ik}+\tilde{l}_{kj}-\tilde{l}_{ij}}$ (using the stored approximations $\tilde{l}$) obeys

$$\tilde{r} = r \cdot 2^{\eta}, \quad |\eta| \leq 3\varepsilon.$$

Since each l has absolute error $\leq \varepsilon$, the sum $l_{ik} + l_{kj} - l_{ij}$ has error $\leq 3\varepsilon$. For sufficiently small ε, $2^{3\varepsilon} = 1 + 3\varepsilon \ln 2 + O(\varepsilon^2)$, so there exists a constant C (e.g., $C = 4$) such that

$$(1 - C\varepsilon)r \leq \tilde{r} \leq (1 + C\varepsilon)r.$$

Summing over all i, j (and noting that the distance condition is checked exactly using the integer matrix M_d, which has no error) gives

$$(1 - C\varepsilon)C_B(k) \leq \tilde{C}_B(k) \leq (1 + C\varepsilon)C_B(k).$$

Choosing $\varepsilon' = \varepsilon/C$ yields relative error at most ε for the final BC values. Rescaling ε concludes the proof.

Remark. The same error analysis applies to stress centrality by replacing $2^{l_{i'k'}+l_{k'j'}-l_{i'j'}}$ with $2^{l_{i'k'}+l_{k'j'}}$ in formula (5).